

Performance Optimization of Lattice Post-Quantum Cryptographic Algorithms on Many-Core Processors

Sandhya Koteshwara, Manoj Kumar, and Pratap Pattnaik

IBM T.J. Watson Research Center

Motivation

- Most public-key cryptography today such as RSA, ECC rely on following “**hard**” problems:
 - a) factoring an integer
 - b) order-finding
 - c) finding discrete logarithm
- Quantum computers (with sufficiently large number of qubits) can solve any of the above three problems with **complexity $O(n^2)$** , i.e. in polynomial time.
- Current users of public-key cryptographic algorithms must ensure that their sensitive data stays indecipherable for many years after traditional approaches are compromised by quantum computing.
- New class of algorithms resilient against attacks by quantum computers are being considered by **NIST for standardization**, collectively known as *post-quantum cryptography (PQC)*. Every platform must become PQC ready!
- Several of the front runners of PQC standardization (***Kyber, Frodo, Dilithium***) are *lattice-based* algorithms.

Lattice-based cryptography

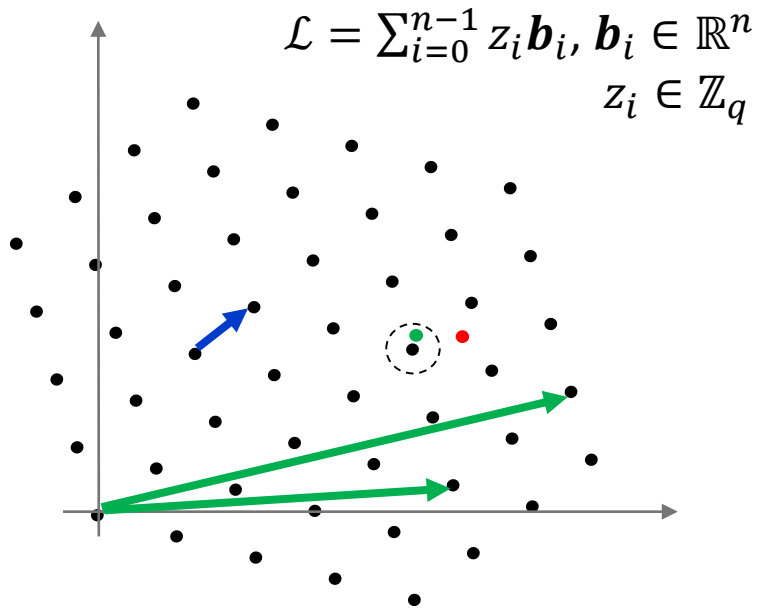


Fig 1: A two dimensional lattice is shown. The two green arrows are the basis. The blue arrow is shorter than γ times the shortest vector. All lattice points are sum of integer multiples of basis vectors. Lattices in PQC have 500 to a few thousand dimensions

- **The hard problem:** Finding the blue vector ($< \gamma \times$ shortest vector), given the green vectors basis, is hard (Known as GapSVP_γ)
- **Formulation:** Learning With Error (**LWE**), used in Frodo
 - The green dot is sum of an error term and a secret combination (**private key**) of the basis
 - Without the private key it is hard to tell the green dots apart from the red dots (randomly chosen point in $\mathbb{R}^n$)
 - The basis vectors can be substituted by a single polynomial with coefficients in $\mathbb{Z}_q$ (**Ring LWE**), or multiple such polynomials (**Module LWE**). Vector-matrix multiplication becomes polynomial multiplication.
- **Characteristics:** Ring and Module LWE (Kyber, Newhope, NTRU, ...)
 - Polynomial multiplication can be expedited using Number Theoretic Transforms (NTT). Biggest time consumer.
 - SHA3 or SHA2/AES used extensively to sample uniform, Gaussian, or centered binomial distributions

Typical lattice PQC workloads

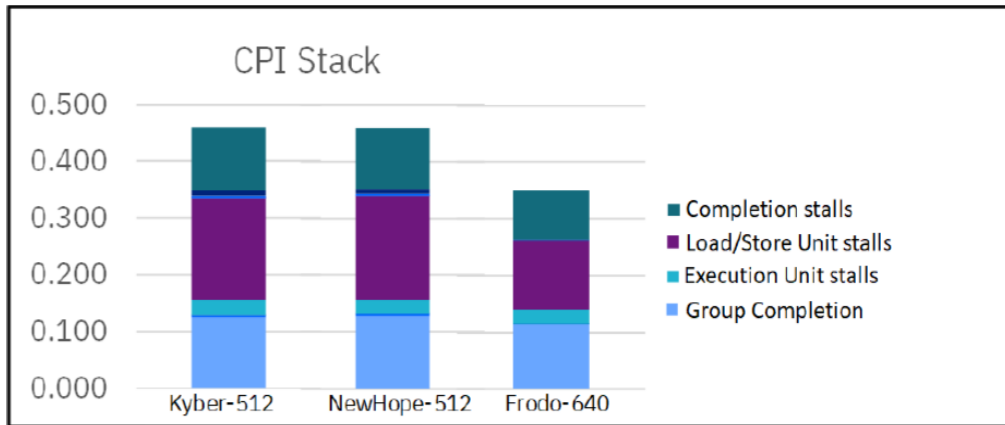


Fig 2. The Cycles-per-instruction (CPI) stack breakdown for lattice-based PQC Kyber, NewHope and Frodo are shown. The values are calculated using Performance Monitor Unit (PMU) counters. Lattice PQC workloads exhibit a CPI of around 0.5 and behave like typical high-performance workloads.

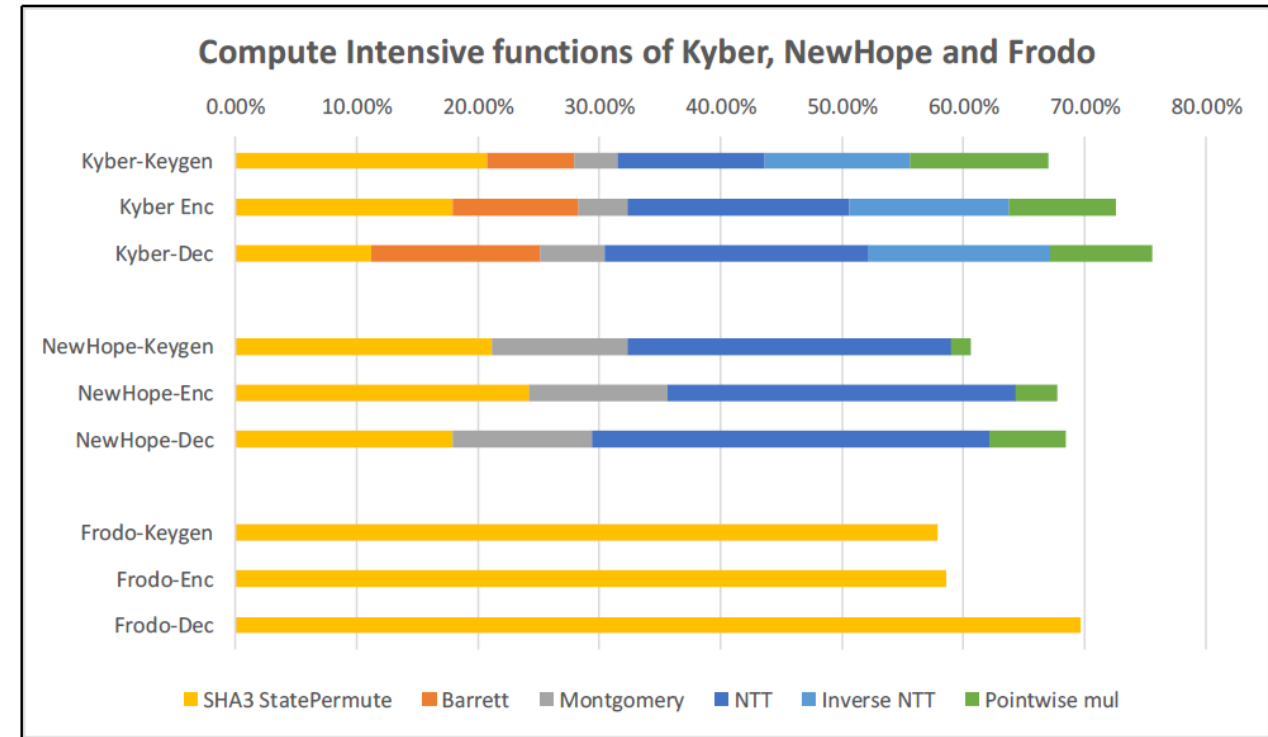


Fig 3. The compute-intensive functions of Kyber, NewHope and Frodo are shown. SHA3 is the most compute intensive function, followed by NTT and Inverse NTT functions. In versions which use AES/SHA2 instead of SHA3, AES and SHA2 will be the most compute intensive.

Optimization strategy: NTT and pointwise multiplication

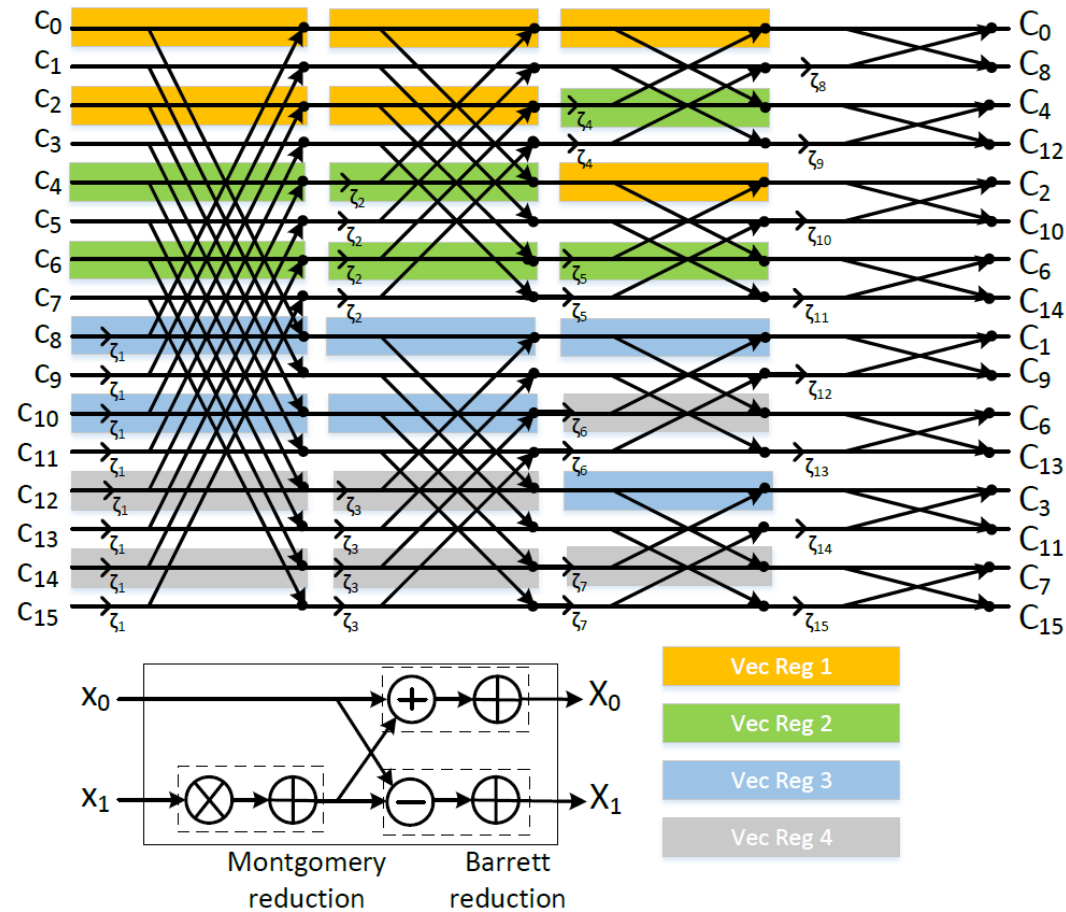


Fig 4. In Kyber, 256 NTT coefficients are processed by loading 8 16-bit coefficients into each 128-bit vector register. In-place computation is performed up to the stage at which coefficients are available. The other half of coefficients are loaded and the computation is repeated. For the last few stages of the algorithm, vector registers are shuffled to align the operands.

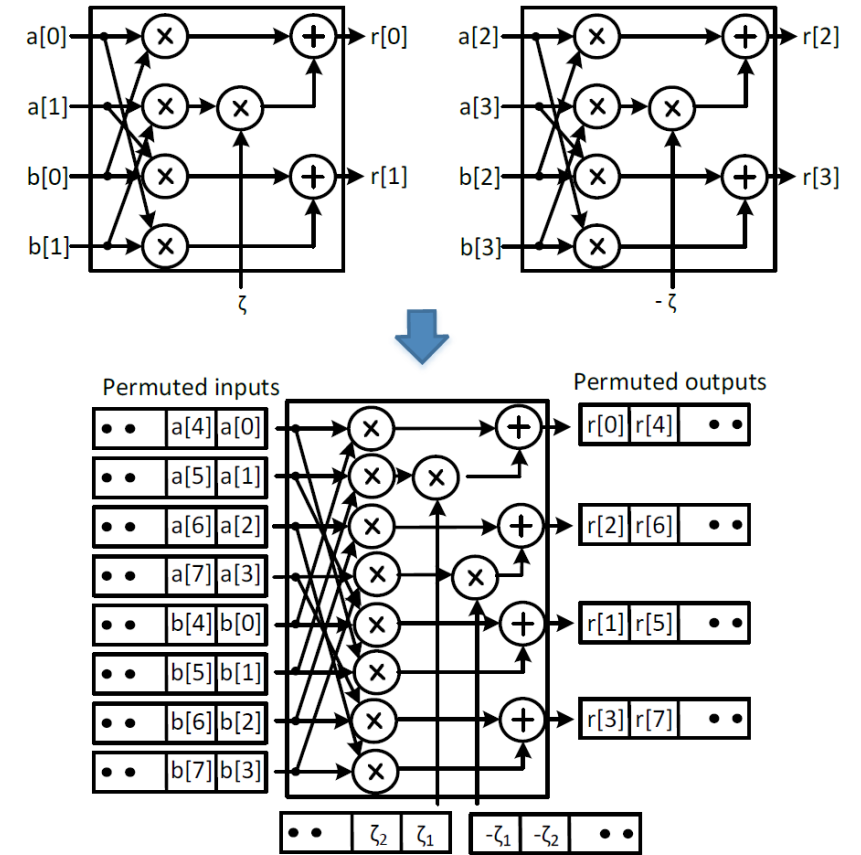


Fig 5. 64 rounds of pointwise accumulation are reduced to 8 rounds by processing 32 coefficients in one loop rather than 4 coefficients in a scalar implementation. The alignment of coefficients is shown in the figure after which 8 vector multiplications and Montgomery reductions can be performed in parallel

Optimization strategy: SHA-3 and AES-CTR

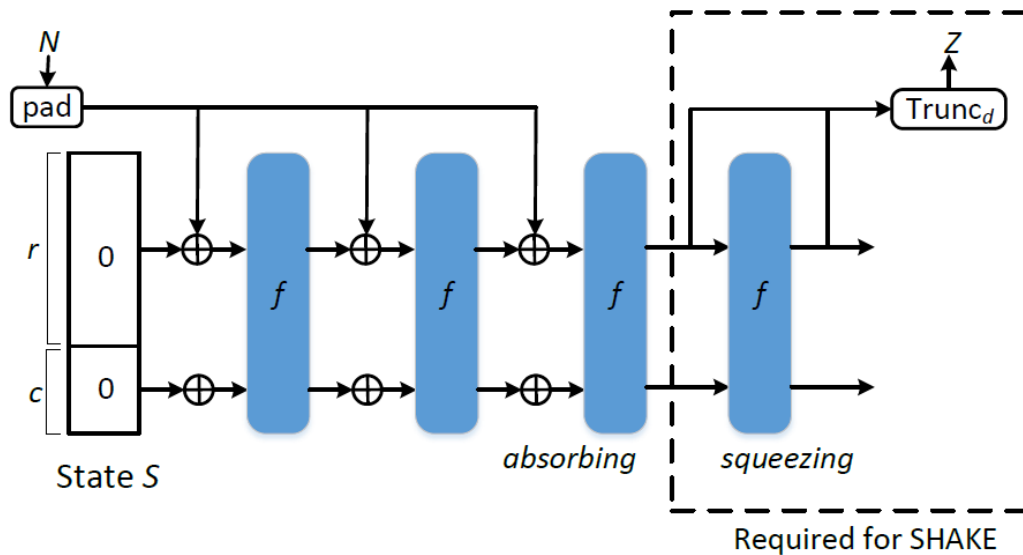


Fig 6. SHA3 algorithm consists of multiple rounds of absorbing and squeezing using state permute function f . However, f cannot be optimized in software. SHA3 can be optimized by parallelly processing multiple streams. Each stream operates on 64 bits of data and a vector register of 128 bits can process 2 streams in parallel.

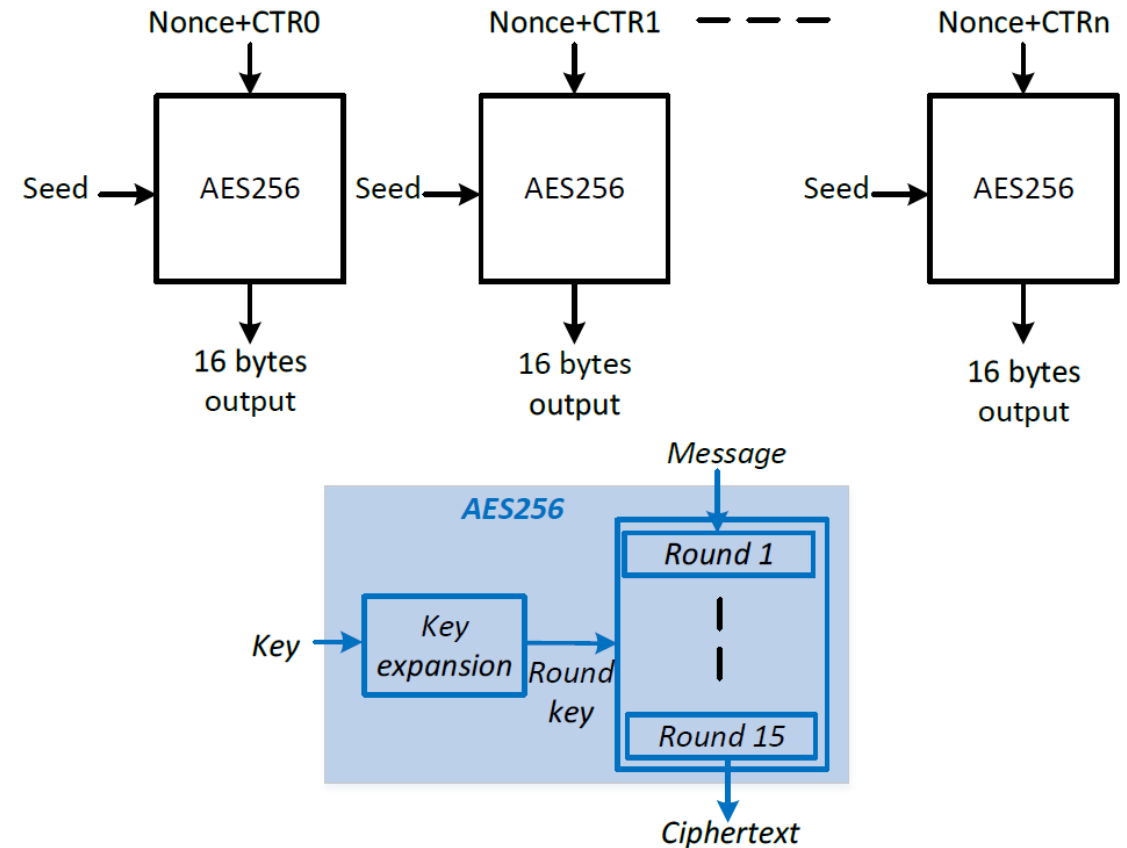


Fig 7. AES-CTR uses multiple blocks of AES in parallel by operating on counter values. Most modern processors have a hardware unit for AES acceleration which can be used for efficient optimization

Performance improvement on Kyber768: Primitives



Fig 8. Performance (cycles/iteration and instructions/iteration) of lattice-based PQC primitives on POWER9 after applying optimizations is shown. The improvement in performance with respect to reference code is around:

- a) 75% for vectorized Montgomery reduction
- b) 90% for vectorized Barrett reduction
- c) 86% for one iteration of vectorized NTT (256 coefficients)
- d) 83% for pointwise multiplication.
- e) 8% improvement for one iteration of StatePermute of SHA3 (Ingestion of 1088 bits)
- f) 95% for generation of 128 bytes of output by AES256 in CTR mode.

Performance improvement of Kyber768

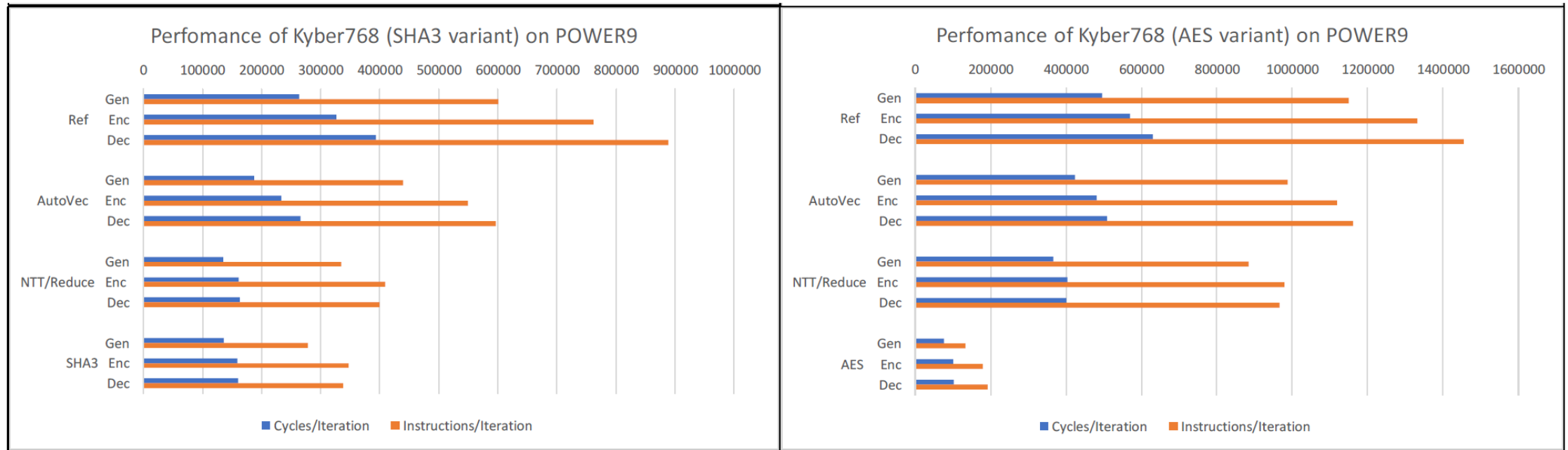


Fig 9. Performance of the reference, auto vectorized, manually vectorized Kyber768-SHA3 variant (NTT+SHA3) and Kyber768-AES/SHA2 variant (NTT+AES) on POWER9 processors is shown.

- Compilation with vectorization flags and inlining of the Montgomery and Barrett reduction functions results in significant improvement in performance, 30%. No further improvements are obtained even after inlining other parts of the code.
- Improvement due to vectorization of NTT is on average 52%.
- NTT vectorization + Vectorization of SHA-3 does not result in any further significant performance improvement.
- NTT vectorization + Vectorization of the AES algorithm in CTR mode with use of AES hardware results in a large reduction of cycles i.e., 83%.
- NTT vectorization + AES acceleration + Vectorization of SHA2 Algorithm is not very effective due to the absence of parallel streams of data.

Kyber768 workload after optimization

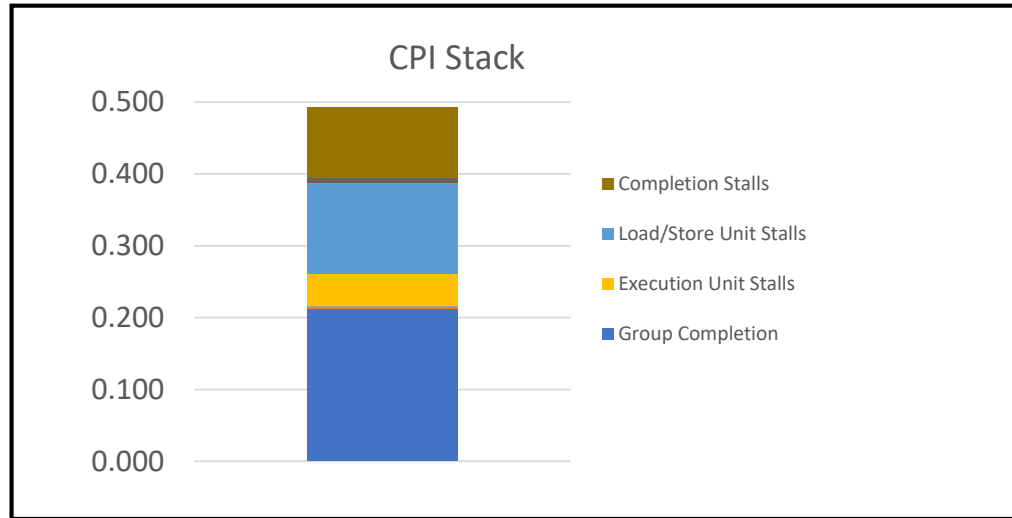


Fig 10. The CPI stack breakdown for Kyber768 (SHA3 variant) after optimization. Note that the CPI remains at around 0.5 which is typical of high-performance workloads. When compared with the unoptimized Kyber implementation, the optimized code exhibits 56%, 68%, 5% and 21% reduction in completion stalls, Load/Store Unit stalls, execution unit stalls, and group completion stalls, respectively.

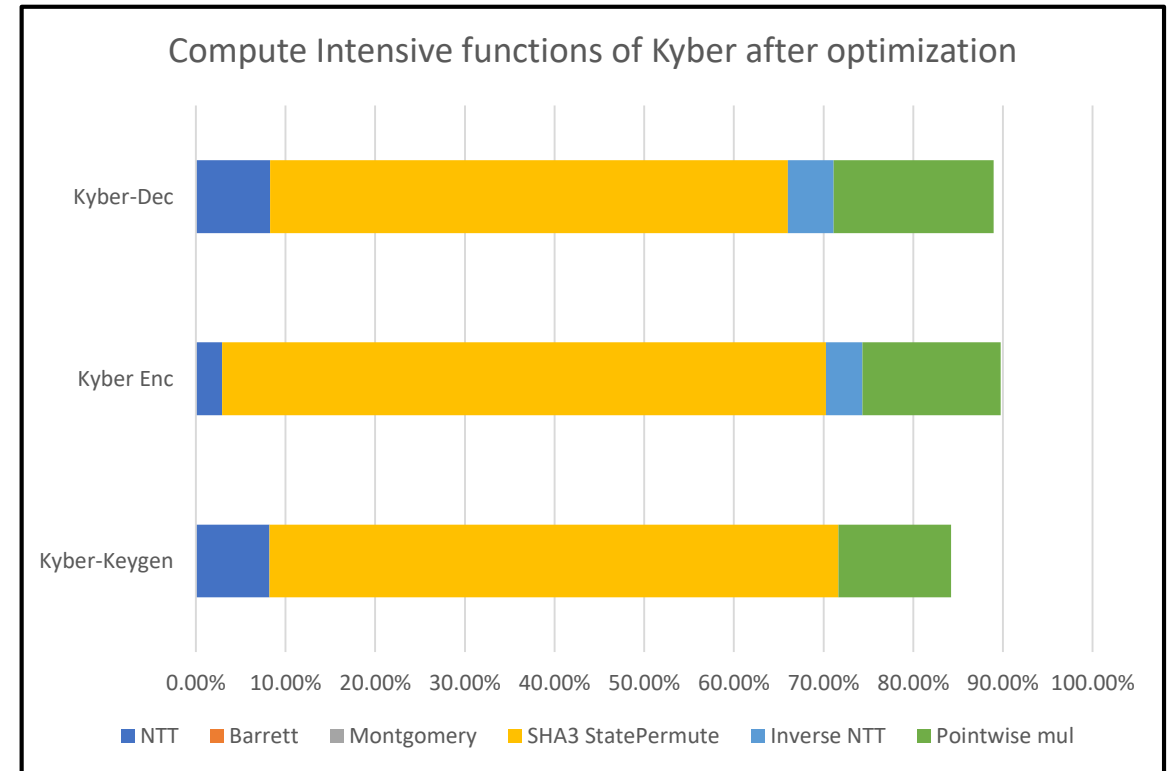


Fig 11. The compute-intensive functions of Kyber after optimization. After successful optimization of polynomial multiplication, SHA3 is the most-compute intensive function. Optimization of SHA3 requires hardware acceleration.

Conclusions and Future work

- Heaviest **time-consuming primitives** of lattice-based cryptographic algorithms are:
 - Polynomial multiplication (Montgomery, Barrett reduction, NTT)
 - Random data generation (SHA3, AES, SHA2).
- Lattice-based PQC algorithms **run as efficiently** as well-behaved scientific engineering code
- **Performance counter analysis** on pre-optimized Kyber code indicated that load/store instructions are the primary reason for CPU stalls. After optimization, the LSU unit stalls are reduced and are almost equal to completion stalls.
- VMX/VSX instruction for improving Kyber768 performance achieves an **improvement of 52%**.
- The importance of hardware acceleration was demonstrated using AES which resulted in **83% improvement** in performance in Kyber768.
- As part of ongoing work, we are working to apply the presented optimizations to **other lattice based PQC**.
- Since the PQC candidate portfolio also includes **code-based and elliptic curve-based** algorithms, future work will involve in-depth analysis of these workloads and their optimization.